

A preliminary study on the impact of a pair design phase on pair programming and solo programming

Matthias M. Müller

Fakultät für Informatik, Universität Karlsruhe, Am Fasanengarten 5, 76 128 Karlsruhe, Germany

Received 26 August 2005; accepted 27 September 2005

Available online 26 January 2006

Abstract

The drawback of pair programming is the nearly doubled personnel cost. The extra cost of pair programming originates from the strict rule of extreme programming where every line of code should be developed by a pair of developers. Is this rule not a waste of resources? Is it not possible to gain a large portion of the benefits of pair programming by only a small fraction of the meeting time of a pair programming session? We conducted a preliminary study to answer this question by splitting the pair programming process into a pair design and a pair implementation phase. The pair implementation phase is compared to a solo implementation phase, which in turn was preceded by a pair design phase, as well. The study is preliminary as its major goal was to identify an appropriate sample size for subsequent experiments. The data from this study suggest that there is no difference in terms of development cost between a pair and a solo implementation phase if the cost for developing programs of similar level of correctness is concerned.

© 2006 Elsevier B.V. All rights reserved.

Keywords: Pair programming; Preliminary study; Post-development test-cases

1. Introduction

Pair programming has become more and more popular in the recent years. Developers who work in pairs in front of one display, keyboard, and mouse learn from their partners and share ideas. Project management appreciates the quantitative benefits of pair programming: faster development and higher quality code. However, the advantages of pair programming are bought at the expense of higher personnel cost, which is balanced only by highly productive pairs [1].

Studies trying to identify the quantitative benefit of pair programming treated the pair programming process as a black box. Williams et al. [2] studied pair programming with 41 undergraduate students. Concerning program correctness, the programs of the pairs passed significant more of the automated post-development test-cases. The data-sample of the pairs also had a smaller variance as opposed to the sample of the individuals. The evaluation of the development cost showed that, after an initial adjustment period in which the pairs spent about 60 percent more programmer hours on the completion of the tasks, the working overhead dropped to 15 percent. Nosek

[3] compared pair programming to solo programming with 15 professional programmers. The experiment task was to implement a database consistency check. All pairs outperformed the individuals. Although the average time for completion was more than 12 minutes longer for individuals, the difference was not statistically significant. Nawrocki and Wojciechowski [4] studied 21 computer science students on the first four assignments of the *Personal Software Process* [5] programming course. Overall, pair programming was not faster than solo programming in sharp contrast to the Williams et al. and the Nosek studies. But the variability within the pair programming group was smaller when compared to the single programmer group. Tomayko [6] compared pair programming to the team software process (TSP) [7]. He reports on a reduced defect density of the programs developed by the team which used pair programming as compared to the team which used the TSP. Heiberg et al. [8] compared pair programming to partner programming. During partner programming, two developers act as a two-person development team. The two developers have to structure the development task into disjoint subtasks such that the work can be done by two programmers in parallel. In their study, Heiberg et al. observed no difference between pair and partner programming concerning cost and product quality. Müller [9,10] conducted two experiments to compare pair programming with solo programming and code reviews. Participants were 38 students. The design of the study forced

E-mail address: muellerm@ipd.uka.de

the subjects to develop programs with a similar number of failures. Both techniques are interchangeable with regards to the development cost. Hulkko and Abrahamson [11] studied the effects of pair programming on product quality during several case studies. They observed *that the relative amount of effort spent on pair programming is at its highest in the beginning of the project and in the defect correction (performed after system test) phase of the project*. Concerning pair programming productivity, Hulkko and Abrahamson report that the productivity of pair programming as compared to solo programming varied between the studied projects. This result complies with the divergent findings of [2–4]. So far, no study has tried to decompose the pair programming process. It might be that the benefits of pair programming can be obtained by only a small fraction of the meeting time of a pair programming session. In order to study this hypothesis, we focus on a pair design session in which a mental model of a potential solution to a problem is built. We distinguish between the following techniques:

- In *Pair Design* two developers work together in order to establish a design of the implementation.
- During *Pair Implementation* two developers work together in front of one display, mouse, and keyboard.
- *Pair programming* is used when we do not want to emphasize the development phase in which the pairing takes place.

In this paper, we present the results of a study analyzing the impact of a pair design session on the cost of pair implementation and solo implementation. In the solo implementation scenario, developers pair off for the design and split up for implementation. To factor out different level of correctness of developed programs, we issued an acceptance-test. The task of the acceptance-test was to ensure a high and similar level of correctness. If a program does not pass the acceptance-test, participants had to correct the failures.

We call the execution of the acceptance-test and the following rework phase the *acceptance-test* phase. Thus, the whole development process in the experiment is split into three phases: the *pair design* phase, the *coding* phase before the first acceptance-test, and the whole *implementation* phase which consists of the coding and the *acceptance-test* phase. With these phases in mind, our study tries to evaluate the following null-hypotheses:

- $H_0(\text{Cost}_{\text{Coding}})$: The cost for coding phase is *not* higher for pair implementation than for solo implementation.
- $H_0(\text{Cost}_{\text{Imp}})$: The cost for the whole implementation is *not* higher for pair implementation than for solo implementation.
- $H_0(\text{Failures})$: The number of failures after coding is *not* smaller for the programs of the pair group than for the programs developed by solo programmer group.

The study was conducted at the University of Karlsruhe in the summer term 2004. Participants were 18 computer science students.

The study is preliminary because we wanted to identify appropriate sample sizes for subsequent experiments and we also wanted to test our newly developed web-based acceptance test system.

2. The study

The experiment had an inter-subject design and was held at the University of Karlsruhe during the summer lectures 2004. The study was part of an extreme programming course [12]. The course was composed of four sessions (introducing pair programming, test-first, refactoring, and the planning game) and a whole week of project work. The experiment took place from May to June after the pair programming introduction. Java and the Eclipse development environment were used for both the experiment and the lab course. The subjects knew from the very first course announcement that they had to take part in an experiment in order to get their course credits. The two groups were called the *pair programming group* (PPG) and the *solo programmer group* (SPG).

2.1. Participants

Eighteen (18) computer science students participated in the study. They had on average 7.5 years programming experience and 3 years programming experience with Java. Although the average is 7.5 years, the actual programming experience is different from student to student. For example, one student already has 16 years programming experience while other students did not program until they started their university studying two years ago. Three subjects tried pair programming prior to the course. Seven subjects reported to have experience with the Eclipse development environment. An introductory course which demonstrated core features of Eclipse was offered to the remaining participants. Pair programming was introduced by extreme programming professionals. The pair programming course took about 1.5 h.

2.2. Experiment task

Students had to design and implement a scheduling algorithm and the elevator control of an elevator system. The elevator system resides in a building with multiple floors and multiple elevators. The system distinguishes between three actions:

- *Request*. An ‘up’ or ‘down’ button outside the elevators is pressed.
- *Job*. The destination floor inside an elevator is chosen.
- *Lock*. The door-lock button delays closing of the elevator door while the door is open.

The task of the scheduling algorithm is to assign *Requests* to an elevator. Specification of the scheduler is kept imprecise as the subjects should not rely on a special strategy as long as every elevator is assigned at least one request during simulation. The elevator control consists of a finite automaton

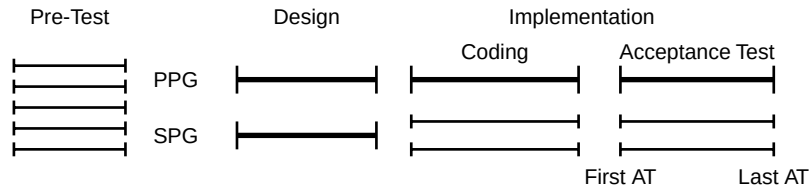


Fig. 1. Experiment plan.

with five states. The behavior of the elevator is described by natural language never mentioning the finite automaton explicitly. The tasks involved implementing two methods residing in the elevator system class and the elevator class.

2.3. Experiment plan

The experiment consisted of three phases: *pre-test*, *design*, and *implementation*. Fig. 1 depicts an overview. Purpose of the *pre-test* was to measure programming skills of participants, which was used later on to divide the students into the groups.

The two remaining phases are dedicated to the elevator task. All subjects worked in pairs for the *design* phase regardless of the group they belonged to. During the design phase, the pairs had to acquaint themselves with the code base and the task itself by the aid of a template, which they had to fill out. At the end of the design, the template was checked for consistency and proper level of detail. Correcting or masking errors was not subject to the check. The pairs had to rework on the template if inconsistencies or a wrong level of detail were found. Checking of the template was done manually by the experimenter. For *implementation*, the pairs of SPG were split off. The individuals worked on their own for the remainder of the experiment. The pairs of PPG proceeded unchanged.

Implementation was divided subsequently into *coding* and *acceptance-test* (AT). During *coding*, the subjects had to implement the task until they thought they had finished. At this point, they entered the *acceptance-test* phase where their programs had to pass the acceptance-test. If the programs did not pass, subjects got the output of the failed tests and had to fix the errors. Acceptance-test and rework repeated as long as the program failed the test. Otherwise, subjects finished the task.

2.4. Calculation of group sizes

The structure of the experimental design and the number of available participants makes it impossible to assure an equal number of experimental units in each treatment group. However, different treatment group sizes reduce power [13]. In our case we have S_{PPG} subjects in the pair programming group which correspond to $n_{PPG} = S_{PPG}/2$ experimental units and S_{SPG} in the solo programming group corresponding to $n_{SPG} = S_{SPG}$ experimental units. Note for pair design comparisons the number of experimental units is $S_{PPG}/2$ and $S_{SPG}/2$ in each group. In order to maximise the chance of revealing an effect, even with a small number of participants, the harmonic mean n is used in the formula [13]

$$n = \frac{2 \times n_{PPG} \times n_{SPG}}{n_{PPG} + n_{SPG}}.$$

The power of the t -test (we use the t -test only for power calculation purposes) increases with the size of the harmonic mean if all other factors (effect size and significance level) are kept constant. The number of data points of both groups must adhere to

$$n_{PPG} \times 2 + n_{SPG} = 18.$$

Table 1 lists the power of the t -test β for different combinations of n_{PPG} and n_{SPG} . The effect size is kept constant at 0.8.

The row with the bold figures marks the combination of n_{PPG} and n_{SPG} with the highest power. In the beginning, we did not expect the power to be of reasonable height and the total increase in power of 0.6 percent as compared to the row above is only of theoretical value. But nevertheless, the application of power calculation for group size estimation in the context of a given and fixed sample size was new to us. Table 2 summarizes group sizes and available data points. We initially planned for 5 data points for PPG and 8 data points for SPG. However, 2 subjects of SPG did not finish coding because of the difficult programming task. The dropouts reduced the number of available data points for SPG by 2.

2.5. Division of subjects into groups and pairs

Division of subjects was done according to their performance in the pre-test. In the pre-test subjects had to write insertion, transformation, and test-methods for a class

Table 1

Power values for different sample sizes of PPG and SPG and constant effect size of 0.8

n_{PPG}	n_{SPG}	N	β
7	4	5.09	0.203
6	6	6	0.240
5	8	6.15	0.246
4	10	5.71	0.229

Table 2

Group sizes and available data points

Group	Size	Pairs in design	Planned data points	Available data points
PPG	10	5	5	5
SPG	8	4	8	6

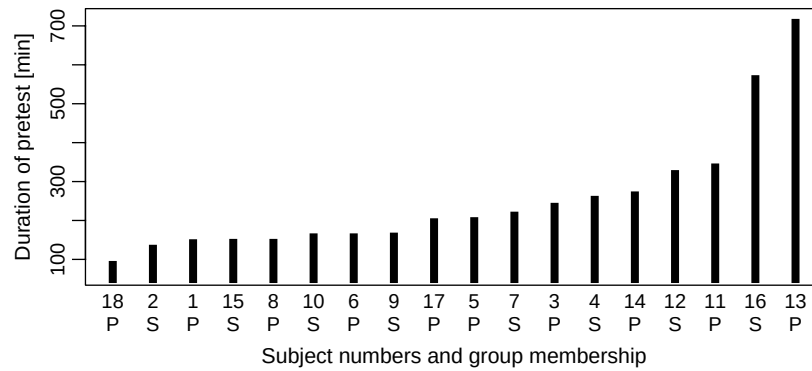


Fig. 2. Length of pre-test and group membership for each subject.

implementing a binary tree. Fig. 2 shows for each subject the length of the pre-test as well as the membership to one of the two groups: P for PPG; S for SPG.

There exist two ways to compose pairs for a pair programming experiment. The first way is to pair off friends. These pairs are expected to be most productive, as they do not have to become acquainted with their partner. However, this selection method requires that every participant is familiar with somebody. If it is not possible to ensure pairs are composed of friends, pairs have to be selected randomly in such a way that every pair is made up of two students who do not know each other. We chose the second method because we had some subjects who reported not to know anybody else in the course.

2.6. Acceptance-test

The acceptance-test is implemented as a web-application. It consists of a simulation environment, which applies several scenarios to the implementation under test. For each scenario, the simulation environment records elevators' state transitions and the issued actions. The subjects had to log in, submit their Java-files, and start the test. If a test fails, subjects obtain the corresponding log-file and a message indicating the type of failure and the time step the failure occurred for the first time.

The acceptance-test consists of 50 scenarios, which try to provoke distinct kind of failures in the implementation under test. Thirty scenarios scrutinize basic elevator functionality by simulating a setting with one elevator. Ten scenarios concentrate on the correct implementation of the door-lock behavior. The remaining scenarios simulate multiple elevators trying to provoke errors in the scheduling algorithm. Finally, the elevator principle is checked for all scenarios.

Another topic concerns the invocation of the acceptance-test. We wanted to ensure that the acceptance-test is executed as seldom as possible. Thus, we decided that only the experimenter should have the right to run an acceptance-test. Therefore, we protected the acceptance-test by a password, which was only known by the experimenter. The students in turn had to ask the experimenter to fill in the password for every acceptance-test they wanted to run. With this procedure, we wanted to increase subject's threshold to issue an acceptance-test. But unfortunately, subjects found a way to issue the acceptance-test on their own because of an error in the

web-application. But as the acceptance-test for one program under test lasts for about 170 s (in the best case), a waiting time of about 3 min was large enough for the subjects not to issue a test after every small code change.

2.7. Gathered data

During the experiment, we measured the length of the phases design T_{Design} , coding T_{Coding} , and acceptance-test T_{Test} . The time for the acceptance-test phase T_{Test} contains the execution time of the acceptance-tests. We compared the effort of the acceptance-test phase including test execution time with the effort of the acceptance-test phase without the test execution time. Beside the fact, that the effort was smaller for the second setting, the difference between data sets did not change. Hence, we decided to use the data sets of the acceptance-test phase for analysis, which include the test execution time.

The measured periods of time are used to calculate the cost of the phase design, coding, and implementation:

$$\begin{aligned}
 \text{Cost}_{\text{Design}}(\text{PPG}) &= 2 \times T_{\text{Design}} & \text{Cost}_{\text{Design}}(\text{SPG}) &= 2 \times T_{\text{Design}} \\
 \text{Cost}_{\text{Coding}}(\text{PPG}) &= 2 \times T_{\text{Coding}} & \text{Cost}_{\text{Coding}}(\text{SPG}) &= T_{\text{Coding}} \\
 \text{Cost}_{\text{Imp}}(\text{PPG}) &= 2 \times (T_{\text{Coding}} + T_{\text{Test}}) & \text{Cost}_{\text{Imp}}(\text{SPG}) &= T_{\text{Coding}} + T_{\text{Test}}
 \end{aligned}$$

Measures concerning the level of correctness of the programs were gathered after the experiment took place, for example, the number of failures for an acceptance-test is calculated using the logs of the acceptance-test environment.

2.8. Construct validity of cost measure

For example Williams et al. [2] and Nawrocki and Wojciechowski [4] conducted their studies to evaluate the advantage of pair programming over solo programming, however, the results of their studies are inconclusive to some extent. The results depend not only on the development methods but also on the number of failures, e.g. Williams et al. report on a difference of observed program failures of 15 percent. The number of failures differ because the assignments were considered completed as soon as the participants (pairs

and individuals) claimed so. Thus, the produced programs naturally differ by the time needed for completion *and* by the number of failures because of subjects' different attitude on when it is beneficial to stop testing and when not. It is methodologically questionable to compare these program versions.

This study tries to balance the number of failures by the additional acceptance-test phase at the end of the implementation process. This acceptance-test phase ensures similar number of failures of subject's programs. Thus, the measured programming effort solely depends on the different implementation methods. However, in order to be consistent with previous studies this paper also presents the confounded results before the acceptance test phase.

2.9. Internal threats

In order to obtain more data points, the pairs of SPG where split off after the design phase and *all* members of SPG had to work on the previous developed design. Thus, during the coding and acceptance-test phases, the solo programmers were not allowed to interact with their pair design partner. However, this limitation does not occur in practice where programmers are used to ask their partner for clarification in the case of problems. Unfortunately, we did not take this situation into account when designing the experiment and thus, we do not know if this problem really occurred.

Choosing the Eclipse development environment for the experiment may have an impact on the outcome of the study because several subjects reported that they had no experience with using Eclipse. But, as these students learned to use Eclipse in the introductory session and the pre-test, the author assumes the impact of Eclipse on the results of the experiment to be negligible.

The dropouts of SPG during coding might lead to a biased data set. The sensitivity analysis which is performed in 3.2, revealed a major impact on the results of the coding phase. The reported difference is by 17 percent due to chance if the dropouts of SPG are taken into account. However, this effect is not as dramatic as one would consider because the main contribution of the study concerns the cost difference after the whole implementation phase. For this data set, the impact of the dropouts is negligible.

Participants of SPG were assigned randomly to pairs. This approach is uncommon in practice where developers pair off in order to combine the best domain knowledge to solve a task. Thus, this strategy may have an impact on the performance of the pairs such as the pairs do not perform as good as they could do. An alternative to the random assignment scheme would have been to pair off the most skilled student with the lowest skilled one and to repeat this pattern on the remaining participants. We applied this scheme in previous studies [9, 10]. The goal of this pattern was to balance the performance of the pairs by the means of their mean programming experience. However, we recognized in a follow up study [14] that the mean programming experience of a pair is a poor predictor of the performance of the pair. This result led us to the conclusion

that we had only the two options: pairing off friends or using a random selection scheme. We used the random selection scheme due to the reason stated in 2.5.

2.10. External threats

All participants were students with almost no pair programming experience. The pairs were also made up of students who did not met before: ad-hoc pairs. Pairing off with an unknown partner results in a pair which is not as productive as it would be after a phase in which the partners get to know each other. Thus, a replication of the study with well-rehearsed pairs may lead to a reduction of pair programming cost resulting in a smaller difference between both groups.

3. Results

We use box plots to visualize the results of the measurements. The boxes within a plot contain 50 percent of the data points. The lower (upper) border of the box marks the 25 percent (75 percent) quantile. The left (right) *t*-bar marks the most extreme data point which is no more than 1.5 times the length of the box away from the left (right) side of the box. Data points are visualized with circles. The median is marked with a thin line. The *M* associated with the dashes marks the mean value within a range of one standard error on each side. Evaluation bases on the Mann–Whitney two-sample test (*p*-level of 0.05).

Appendix lists the original data sets for the cost analysis, see Table A1, and the failure frequencies of the developed programs, see Table A2.

3.1. Cost of design phase

Before we analyze the data with respect to the hypotheses, we explore whether the groups have any differences concerning the cost of the pair design phase. The purpose of this comparison is to evaluate whether there is a bias in our data set. As we divided subjects' into groups by the pre-test productivity figures, we expect no difference between the groups. Fig. 3 shows the cost for design phase $Cost_{Design}$ for both groups.

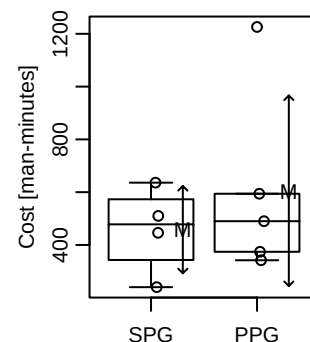


Fig. 3. Cost for pair design phase in man-minutes.

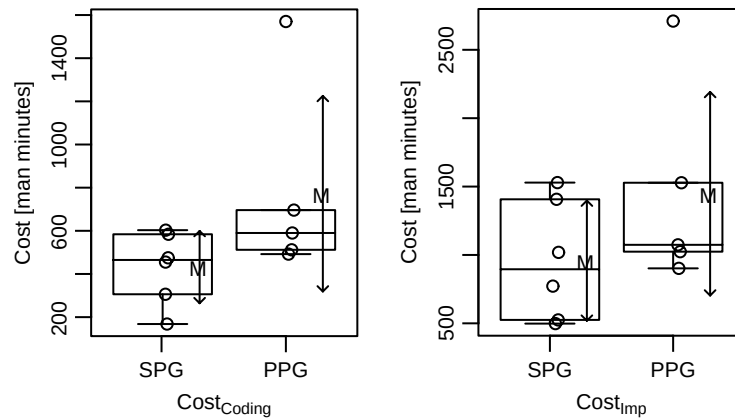


Fig. 4. Cost for coding and implementation without dropouts of SPG.

The boxes of SPG and PPG have an almost perfect match. The medians of the groups are 478 man-minutes (mm) for SPG and 490 mm for PPG. The difference between the means (458 mm for SPG vs. 605 mm for PPG) is due to the outlier of 1226 mm in PPG. The mean for PPG without the outlier is 450 mm. The outlier of PPG originates from a pair where one student rated the complexity of the programming task with 2 while the other student rated it as 5 on a five-point scale ranging from 1 (very easy) to 5 (very difficult). This pair needed also more time for the implementation than the other pairs, as we will see in Section 3.2. As a result from the analysis of the design cost, division of subjects into groups led to groups with almost equally skilled pairs.

3.2. Comparison of cost

The cost of the coding phase and the whole implementation phase is considered. The left plot of Fig. 4 shows the cost for the coding phase $\text{Cost}_{\text{Coding}}$ without the dropouts of SPG.

Half of the single developers in SPG were cheaper than the cheapest pair of PPG. The difference between the data sets is supported by a p -value of 0.041 for the one-sided Mann–Whitney test which leads to a rejection of hypothesis $H_0(\text{Cost}_{\text{Coding}})$ at the 5 percent level. The medians of the groups are 465 mm for SPG and 590 mm for PPG (means: 432 mm SPG vs. 772 mm PPG). Comparing the

medians, SPG is about 21 percent cheaper. Calculating the on average advantage of SPG compared to PPG is meaningless because the outlier in PPG has too large an impact on the mean of PPG for a comparison to be fair. If we had omitted the outlier, the on average advantage would have been 32 percent. This advantage is not significant on the 5 percent level ($p=0.086$) any more. In this case, hypothesis $H_0(\text{Cost}_{\text{Coding}})$ can not be rejected. The observed difference is not caused by the different implementation techniques alone but also due to the different number of failures as it is discussed in Section 3.3. The right plot of Fig. 4 shows the cost of the implementation phase Cost_{Imp} . The cost difference between both groups is not as large as for the design phase. The smaller difference is also supported by a higher p -value of the Mann–Whitney test of 0.123 as compared to the p -value (0.041) for the coding phase. This difference is even smaller if the outlier of PPG is omitted from the data set. The p -value increases to 0.238.

In either case, hypothesis $H_0(\text{Cost}_{\text{Imp}})$ can not be rejected at the 5 percent level. The cost analysis of the implementation phase compares the effort needed to develop programs with similar number of failures. Thus, the observed difference is mainly caused by the different implementation methods. Although the right plot of Fig. 4 shows a trend to favor SPG over PPG data sets are too small for a conclusive result.

In order to study the impact of the two outliers of SPG on the comparison, a sensitivity analysis is performed. The two

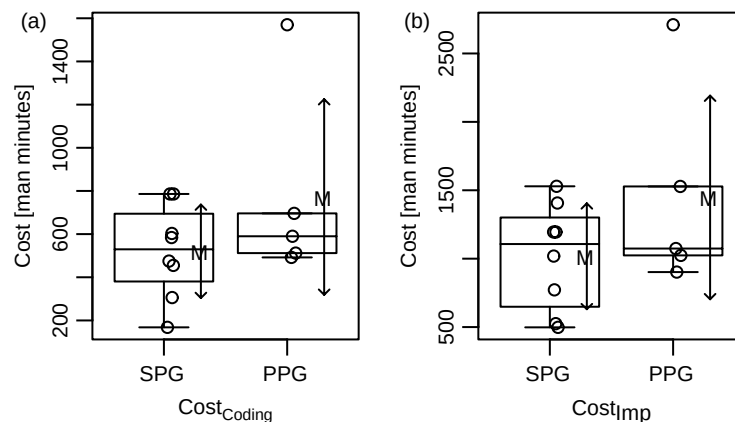


Fig. 5. Cost for coding and implementation with dropouts of SPG.

Table 3
Mann–Whitney one-sided p -values for data sets

Data set	Without SPG dropouts		With SPG dropouts with PPG outlier
	With PPG Outlier	Without PPG outlier	
Cost _{coding}	0.041	0.086	0.170
Cost _{Imp}	0.123	0.238	0.210

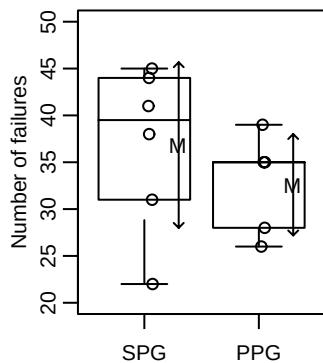


Fig. 6. Number of program failures at first acceptance test.

participants dropped out during coding. Thus, we have to assign the dropouts appropriate values for the time for coding and for quality assurance. They were assigned the maximum value an experimental unit spent for coding (786 min) and the average time for quality assurance (409 min). Fig. 5 shows the two data sets. The SPG data set for the cost of coding has shifted to the upper end of the scale (median: 530 mm, mean: 520 mm).

The mean and the median of SPG have also increased for the implementation cost, although the data points of the dropouts are not the maximum values any more. The data points of the dropouts lie in the third quartile of the data sample. Table 3 summarises the p -values for the Mann–Whitney test of the different SPG and PPG data sets. The last column shows the Mann–Whitney p -values calculated using the SPG data set with the two dropouts (SPG_{dropout}). The difference between SPG_{dropout} and PPG has reduced for the coding phase; see the larger p -value in the last column of the Cost_{Coding} row as compared to the other two p -values in the same row. The difference for the implementation phase has almost kept the same. The p -value changed from 0.238 (SPG vs. PPG, middle column of Table 3) to 0.210 (SPG_{dropout} vs. PPG). We conclude that the dropouts have a major impact on the cost analysis of

the coding phase while the impact on the whole implementation phase is negligible.

Cost analysis lead to the following two results

If level of correctness of developed programs is of no concern, solo programming can be cheaper than pair programming. Comparison of the medians of both groups yields a 21 percent cost reduction of solo programming as compared to pair programming. Hypothesis $H_0(\text{Cost}_{\text{Coding}})$ can only be rejected if the outlier of PPG is taken into account and the two outliers of SPG are ignored. In all other cases, hypothesis $H_0(\text{Cost}_{\text{Coding}})$ has to be rejected.

If programs of similar level of correctness have to be developed, there might be no difference between both groups. Hypothesis $H_0(\text{Cost}_{\text{Imp}})$ can not be rejected regardless whether the dropouts are taken into account or not.

The first result is not only in line with our expectations it complies also to the results of others [2–4] who compared solo programming with pair programming. However, if dropouts of SPG are taken into account, the difference between both groups after coding is negligible. The second result may be caused by the small power of the test or because there is no difference. We should repeat this study with a larger data sample.

3.3. Comparison of failures

Fig. 6 shows the number of program failures after coding. The box containing 50 percent of the data points of PPG is shifted towards the lower end of the SPG box. The medians of both groups are 40 failures per program (fpp) for SPG and 35 fpp for PPG (means: 37 fpp for SPG vs. 33 fpp for PPG). When we take the medians into account the programs of SPG show 14 percent more failures than the programs of PPG. The one-sided Mann–Whitney test reveals a p -value of 0.157. Thus, hypothesis $H_0(\text{Failures})$ can not be rejected at the 5 percent level.

Instead on focusing only on the number of failures, we were also interested in the different types of failures. The barplot of Fig. 7 shows program failures categorized by failure type. Table 4 describes the used failure codes. As the acceptance test contained 50 different scenarios each program could contribute 50 failures to the data set, in the worst case. The first number of the failure code indicates the severity of the failure in ascending order. While a leading 0 indicates minor failures, failures starting with a 3 indicate an application crash. Looking at the barplot, the programs of SPG seem to have problems with the door timer. Five out of 6 programs of SPG

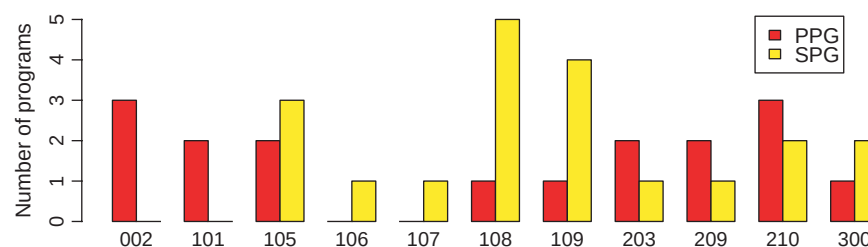


Fig. 7. Comparison of type of program failures for PPG and SPG

Table 4
Failure classification

Failure	Description
002	Job finished too late
101	Illegal speed; job finished too early
105	All elevators halted although jobs exist
106	Elevator principle violated
107	Elevator's floor number changed in the direction opposite to the driving direction
108	Elevator too short in state OPEN
109	Elevator too long in state OPEN
203	Illegal floor number; floor number does not exist
209	Job with illegal state
210	Door lock must not be assigned in state LOCKED
300	Corrupt log file

close the elevator doors too early; see failure 108. Four programs of SPG close the elevator doors too late; see failure 109. The programs of PPG seem to have more problems with the driving timer. Either the elevator waited for too long between a floor change-over (failure 002) or floor change-over was too fast (failure 101). 2 programs of SPG and 1 program of PPG crashed.

We also studied the frequencies the different kinds of failures occur. The barplot of Fig. 8 shows the data sets. Two programs of SPG crashed 67 times, see the SPG bars above failure 300 of Figs. 7 and 8. Five programs of SPG lead to 83 failure situations where the elevator door is closed too early. The PPG programs show failure 105 (elevators halted although jobs exist) and 203 (illegal floor number) most often. To conclude this analysis, solo programmers seemed to have most

problems with the timing of the elevator doors. Failures of the programs developed by the programmer pairs seem to be caused by problems concerning the movement of the elevators.

We also studied whether the programs of the solo programmers who paired off for the design phase show the same types of failures. Fig. 9 shows the comparison of pair 7 and pair 9. The other two pairs of SPG could not be analyzed because these were the pairs where one subject did not finish coding. The failures of the programs developed by the subjects of pair 9 are rather different. But subjects of pair 7 seem to share a wrong mental model because their programs contribute to all program crashes of SPG.

To sum up, programmer pairs tend to develop programs, which show fewer numbers of failures. However, hypothesis $H_0(\text{Failures})$ can not be rejected at the 5 percent level.

Concerning the type of failures shown by our data set, programs developed by programmer pairs might show different kind of failures than the programs developed by the solo programmers. The failures of the SPG programs are formally more severe than the failures of the PPG programs. However, moving an elevator to a non-existing floor is harmful, as well. And last, developers' mental model built in the pair design session has a positive as well as a negative impact on the developed program. And again, the results of this analysis have to be seen in the light of a small data sample. Further studies with a larger sample size should yield more conclusive results. Further studies should also concentrate on the defects and the reason why these defects have been introduced into the program. This analysis would lead to a deeper understanding of the difference between pair and solo programming.

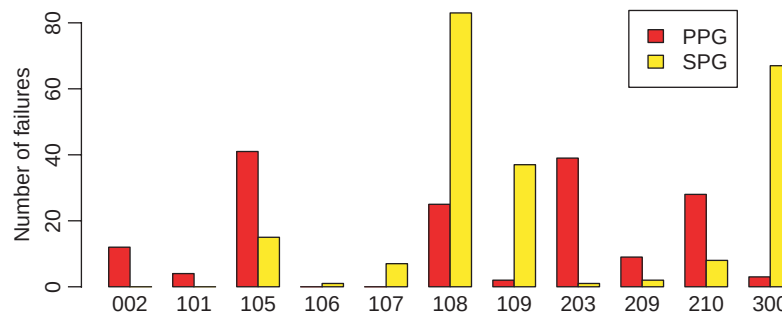


Fig. 8. Failure frequency comparison for PPG and SPG.

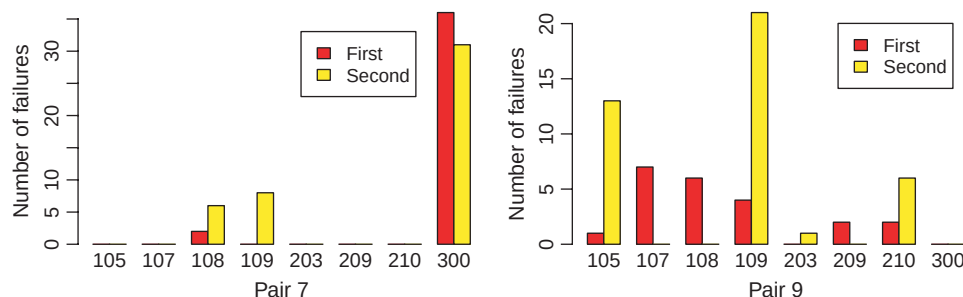


Fig. 9. Failure frequency comparison after coding for solo developers who paired off for the design phase.

Table 5
Effect size ES and sample size n values (ES,n)

	SPG	SPG _{dropout}
PPG	(0.79, 21)	(0.76, 23)
PPG _{no outlier}	(0.46, 59)	(0.33, 113)

3.4. Sample size calculation

Another incentive of this preliminary study was to estimate the expected effect size for the main study, which is planned to be conducted with professional engineers. The effect size is used to calculate the sample sizes of PPG and SPG of the main study. Sample size calculation bases on the parameters power β , significance level α , and effect size ES . We assume standard values for the power and the significance level, i.e. $\beta=0.8$ and $\alpha=0.05$. In order to estimate the number of professionals needed for the study, the effect size has to be calculated from our data set using the equation

$$ES = \frac{\overline{\text{Cost}_{\text{Imp}}(\text{PPG})} - \overline{\text{Cost}_{\text{Imp}}(\text{SPG})}{s}$$

where $\overline{\text{Cost}_{\text{Imp}}}$ denotes the average cost of the implementation phase and s is the standard deviation of the pooled data sets. Table 5 shows effect sizes and sample sizes for different combinations of PPG and SPG data sets.

For example, the table entry of PPG_{nooutlier}/SPG denotes an effect size of 0.46 which is almost as large as a medium-sized effect (0.5) of Cohen's [13] effect size measures. Using power tables of the one-sided two-sample t -test we obtain 59 data points per group. If the Mann–Whitney test instead of the t -test is used for evaluation, group sizes have to be adjusted to account for the 13.4 percent weaker asymptotic relative efficiency of the Mann–Whitney test as compared to the t -test [15]. Thus, the previous calculated sample size has to be divided by 0.864, which leads to a group size of 69 data points. Finally, if a study should have a chance of 80 percent to detect with the t -test a cost difference between PPG and SPG for the implementation phase on a 5 percent level, the groups should be made up of 118 participants for PPG and 59 subjects for SPG.

The question arises whether the estimated sample size which was calculated using a group of students can be used for a target population of professionals, as well. The problem is that the effect size depends on the differences of the group means and the variability of the data samples. If we assume that the difference between the sample means remains the same but the variability in the data sample changes, then the group sizes which are necessary for a hypotheses test to have the same power changes also. For example, if the data samples of the professionals expose a larger variability than the students sample, the effect size would decrease. But, a smaller effect size would mean that we have to use a larger data sample for a hypothesis test in order to have the same power. However, there exists no study stating that groups, which are made up of pairs of professionals expose a larger or smaller variability in performance than groups, which are made up of pairs of

students. Thus, it is not possible to adjust the previous calculated sample size values. But, if we perform a longitudinal study over several weeks where we collect the data step by step, we can apply sequential tests to see whether it is reasonable to stop collecting data or not, see for example [16].

4. Conclusions

We presented the results of a study, which analyzed the impact of pair design on the cost of pair implementation and solo implementation. The study was conducted at the University of Karlsruhe with 18 computer science students. Our data set suggests that solo programming is as expensive as pair programming if programs of similar level of correctness have to be developed. Hypothesis $H_0(\text{Cost}_{\text{Coding}})$ could be rejected only in the for the solo programming group most optimistic data setting. The two other hypotheses $H_0(\text{Cost}_{\text{Imp}})$ and $H_0(\text{Failures})$ could *not* be rejected.

The small data might be the reason for finding no difference between both methods. To increase the likelihood of revealing an effect to 80 percent the study should be repeated using 59 data points per group.

We also analyzed the type of program failures of the developed programs. Solo programmers seem to have different problems than developer pairs. Naturally, this result is preliminary due to the small data set and it should be based on an analysis of program faults rather than on the observed program failures.

Concerning development cost, our data suggests a pair design phase to be a potential alternative to the expensive pair

Table
A1 Duration of experiment phases

PNR	PairNR	Paired	Partner No	T_{design}	T_{coding}	$T_{\text{coding}} + T_{\text{test}}$	Drop-out
1, 13	1	Yes	1	297	348	451	no
17, 18	2	yes	1	171	246	537	no
6, 11	3	yes	1	245	256	764	no
3, 8	4	yes	1	187	295	512	no
5, 14	5	yes	1	613	785	1355	no
4	6	No	1	223	475	772	no
10	6	No	2	223	786	1195	yes
15	7	No	1	255	168	525	no
16	7	No	2	255	455	1019	no
2	8	No	1	318	306	498	no
12	8	No	2	318	786	1195	yes
7	9	No	1	120	584	1529	no
9	9	No	2	120	603	1407	no

Table
A2 Description of the acronyms of Table 6

PNR	Number of participant
PairNR	Number of pair during design, for PPG also for implementation
Paired	Data point originates from a pair
Dropout	Participant did not finish the experiment

Table
A3 Program failures

PNR	E002	E101	E105	E106	E107	E108	E109	E203	E209	E210	E300
1,13	0	0	0	0	0	25	0	0	0	0	3
17,18	3	0	18	0	0	0	0	18	0	0	0
6,11	6	3	0	0	0	0	2	0	7	8	0
3,8	3	0	23	0	0	0	0	0	0	9	0
5,14	0	1	0	0	0	0	0	21	2	11	0
4	0	0	0	0	0	40	4	0	0	0	0
15	0	0	0	0	0	2	0	0	0	0	36
16	0	0	0	0	0	6	8	0	0	0	31
2	0	0	1	1	0	29	0	0	0	0	0
7	0	0	1	0	7	6	4	0	2	2	0
9	0	0	13	0	0	0	21	1	0	6	0

programming process. However, a pair design phase does not solve every problem as even two developers are not immune to a wrong mental model. But the probability of building a wrong solution might be much lower for a developer pair than for a single programmer.

Acknowledgements

The author would like to thank Barbara Kitchenham and the anonymous reviewers for her remarks, Henning Brechmacher for supervising the students during the experiment, and Guido Malpohl for proofreading a previous version of this article.

Appendix A. Appendix data sets

Tables A1 and A3 show original data sets. Table A2 lists the descriptions of the column labels of Table A1. The meaning of the failure codes of Table A3 is given by Table 4.

References

- [1] F. Padberg, M. Müller, in: International Symposium on Software Metrics (Metrics), Sydney, Australia, 2003, pp. 166–177.
- [2] L. Williams, R. Kessler, W. Cunningham, R. Jeffries, IEEE Software (2000) 19.
- [3] J. Nosek, Communications of the ACM 41 (1998) 105.
- [4] J. Nawrocki, A. Wojciechowski, in: European Software Control and Metrics (Escom), London, UK, 2001.
- [5] W. Humphrey, A Discipline for Software Engineering, Addison-Wesley, Reading, MA, 1995.
- [6] J. Tomayko, Computer Science Education 12 (2002) 213.
- [7] W. Humphrey, Introduction to the Team Software Process, Addison-Wesley, Reading, MA, 1999.
- [8] S. Heiberg, U. Puus, P. Salumaa, A. Seeba, in: XP 2003, in LNCS, vol. 2675, Springer, Berlin, 2003, pp. 215–224.
- [9] M. Müller, Journal on Empirical Software Engineering 9 (2004) 335.
- [10] M. Müller, Journal of Systems and Software (JSS) 78 (2005) 166.
- [11] H. Hulkko, P. Abrahamsson, International Conference on Software Engineering, St Louis, Missouri, USA, 2005, pp. 495–503.
- [12] M. Müller, J. Link, R. Sand, G. Malpohl, International Conference on Extreme Programming and Agile Processes in Software Engineering (XP2004), No. 3092 Lecture Notes in Computer Science, Springer, Garmisch-Partenkirchen, Germany, 2004, pp. 294–302.
- [13] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, Academic Press, New York, 1988.
- [14] Müller, M., Padberg, F. International Symposium on Software Metrics (Metrics), Chicago, IL, USA, 2004, pp. 151–158.
- [15] M. Hollander, D. Wolfe, Nonparametric Statistical Methods, second ed, Wiley, New York, 1999.
- [16] B. Ghosh, Sequential Tests of Statistical Hypotheses, Addison-Wesley, Reading, MA, 1970.